



AL MAAREF UNIVERSITY

Mastering Life with Wisdom & Knowledge

Welcome to HackathonTech 2026

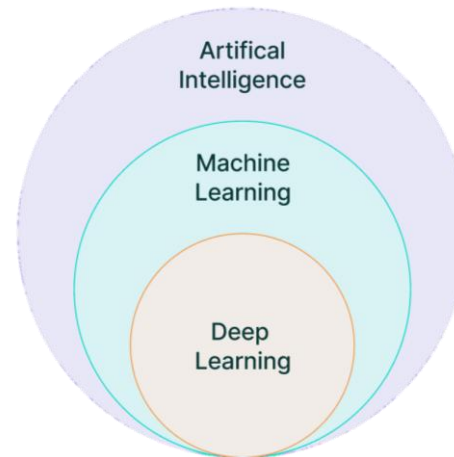
Part 3: **Machine Learning**



What is AI

The goal of AI is to perform tasks that help reduce repetitive work and make tasks easier.

Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.

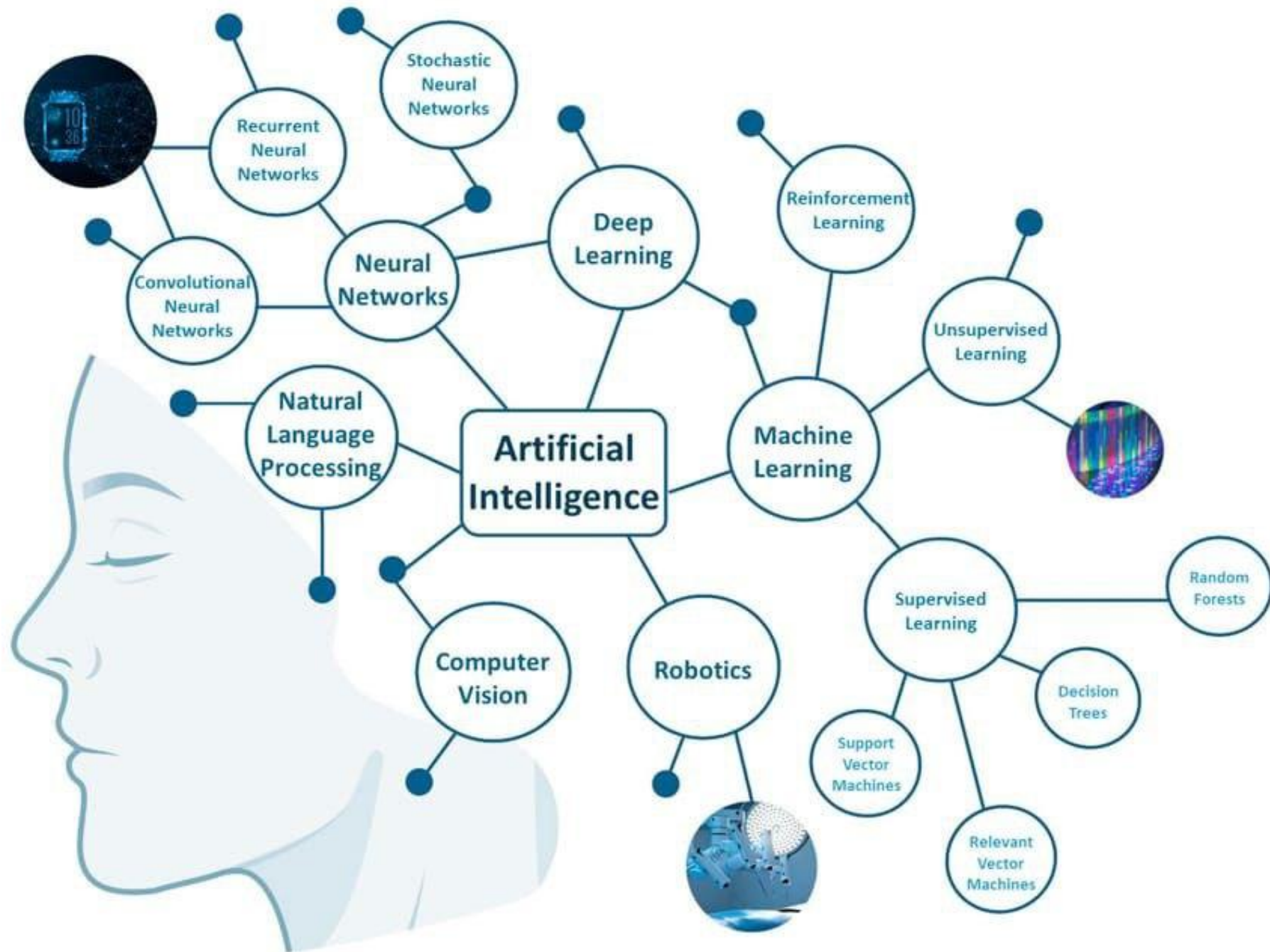


Why ML

Our aim is to begin our journey in AI, so we chose Machine Learning as our starting point.



is an important subset of artificial intelligence. ML is concerned with using specialized algorithms to uncover meaningful information and find hidden patterns from perceived data to corroborate the rational decision-making process.





Analogy between the human brain and ML



senses

Experience

Rule
formation

Pattern
recognition



Machine Learning

Learn by finding patterns in data

Make a
decision



writing Rules

Does it have fur?
Is it small?
Does it live inside the house?
Does it say Meow?

We did not
guess
randomly

we used rules +
Information

It is a Cat



AI Does the same thing

In ML

Instead of a human asking questions,
a computer looks at data.

Input (features):

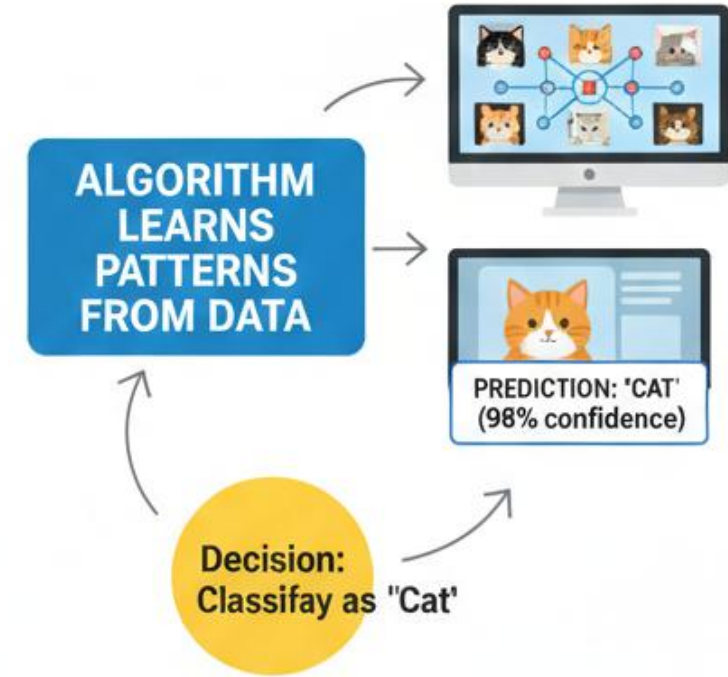
Fur = yes

Size = small

Sound = meow

Lives at home = yes

Output:
Cat 🐱



It is:

A mathematical representation

Learned from data

Used to make predictions

What is a Model?

**A model is the final result
of learning**

Example:

**A trained model can predict
house prices based on size and
location**

**In ML:
The algorithm learn patterns
from data**

What is Algorithm?

**An algorithm is a step by
step procedure used to
solve a problem**

**Example:
Linear regression, KNN,
Decision Tree**

Think of it as the learning strategy

What is Dataset?

**A dataset is a structured
collection of data**



**Example:
A table with
Rows -> observations
Columns -> features (attributes)**

Features

	A	B	C	D	E	F	G	H	I
1	gender	race_ethn	parental_l	lunch	test_prep	math_sco	reading_s	writing_score	
2	female	group B	bachelor's	standard	none	72	72	74	
3	female	group C	some colle	standard	complete	69	90	88	
4	female	group B	master's d	standard	none	90	95	93	
5	male	group A	associate'	free/redu	none	47	57	44	
6	male	group C	some colle	standard	none	76	78	75	
7	female	group B	associate'	standard	none	71	83	78	
8	female	group B	some colle	standard	complete	88	95	92	
9	male	group B	some colle	free/redu	none	40	43	39	
10	male	group D	high schoo	free/redu	complete	64	64	67	
11	female	group B	high schoo	free/redu	none	38	60	50	
12	male	group C	associate'	standard	none	58	54	52	
13	male	group D	associate'	standard	none	40	52	43	
14	female	group B	high schoo	standard	none	65	81	73	
15	male	group A	some colle	standard	complete	78	72	70	
16	female	group A	master's d	standard	none	50	53	58	
17	female	group C	some high	standard	none	69	75	78	
18	male	group C	high schoo	standard	none	88	89	86	
19	female	group B	some high	free/redu	none	18	32	28	
20	male	group C	master's d	free/redu	complete	46	42	46	
21	female	group C	associate'	free/redu	none	54	58	61	
22	male	group D	high schoo	standard	none	66	69	63	

Data

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

labels

Customer Id	Age	Edu	Years Employed	Income	Card Debt	Other Debt	Address	DebtIncomeRatio
1	41	2	6	19	0.124	1.073	NBA001	6.3
2	47	1	26	100	4.582	8.218	NBA021	12.8
3	33	2	10	57	6.111	5.802	NBA013	20.9
4	29	2	4	19	0.681	0.516	NBA009	6.3
5	47	1	31	253	9.308	8.908	NBA008	7.2
6	40	1	23	81	0.998	7.831	NBA016	10.9
7	38	2	4	56	0.442	0.454	NBA013	1.6
8	42	3	0	64	0.279	3.945	NBA009	6.6
9	26	1	5	18	0.575	2.215	NBA006	15.5
10	47	3	23	115	0.653	3.947	NBA011	4
11	44	3	8	88	0.285	5.083	NBA010	6.1
12	34	2	9	40	0.374	0.266	NBA003	1.6

unlabeled

Now, Guess which one is Supervised or Unsupervised

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

labels

Supervised

Customer Id	Age	Edu	Years Employed	Income	Card Debt	Other Debt	Address	DebtIncomeRatio
1	41	2		6	19	0.124	1.073 NBA001	6.3
2	47	1		26	100	4.582	8.218 NBA021	12.8
3	33	2		10	57	6.111	5.802 NBA013	20.9
4	29	2		4	19	0.681	0.516 NBA009	6.3
5	47	1		31	253	9.308	8.908 NBA008	7.2
6	40	1		23	81	0.998	7.831 NBA016	10.9
7	38	2		4	56	0.442	0.454 NBA013	1.6
8	42	3		0	64	0.279	3.945 NBA009	6.6
9	26	1		5	18	0.575	2.215 NBA006	15.5
10	47	3		23	115	0.653	3.947 NBA011	4
11	44	3		8	88	0.285	5.083 NBA010	6.1
12	34	2		9	40	0.374	0.266 NBA003	1.6

unlabeled

Unsupervised

Supervised

We have correct answer (label)

Unsupervised

Cluster the data into groups of similar features



Supervised Learning

Part of ML that uses labeled datasets to train algorithms to predict outcomes and recognize patterns.

Labeled Data



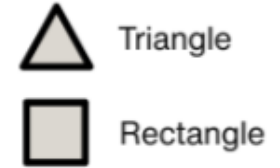
Machine



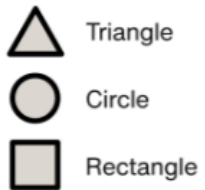
ML Model



Predictions



Labels



Test Data





Regression and Classification

is a way where the computer learns to predict a numerical value

Input: continuous
Output: continuous

Example: What will be the temperature tomorrow => 3C, 5C

is a way where the computer learns to categorize data

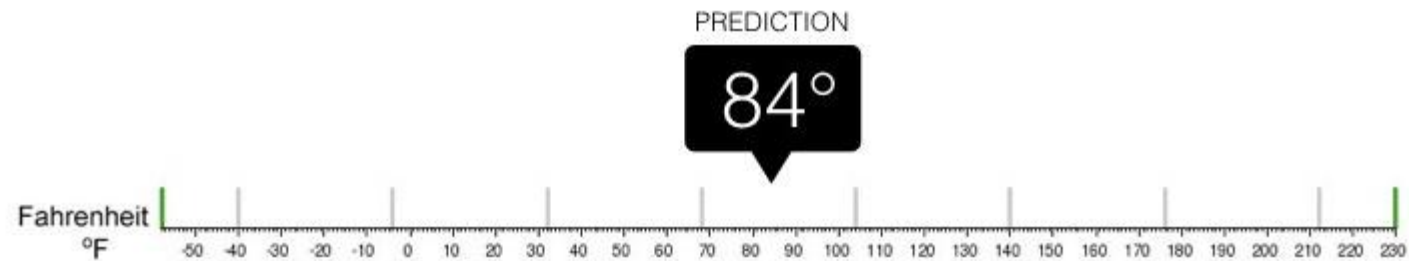
Input: discrete
Output: discrete

Example: will it be hot or cold tomorrow=> hot or cold



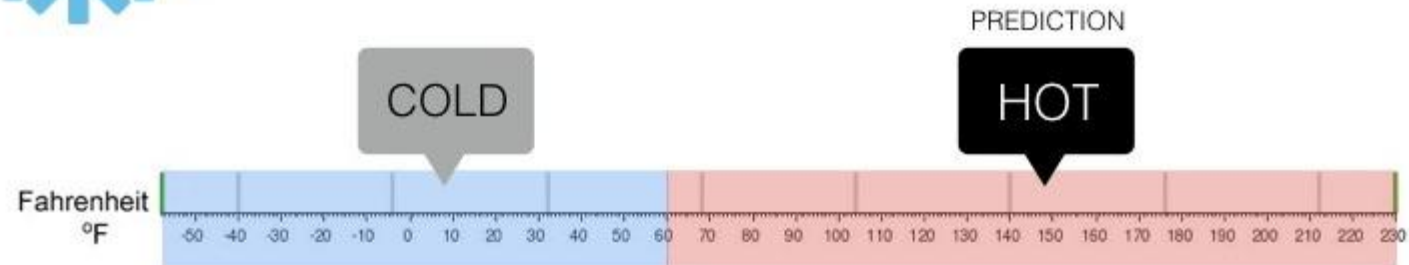
Regression

What is the temperature going to be tomorrow?



Classification

Will it be Cold or Hot tomorrow?





Models



Simplest Algorithm: Linear Regression

→ Regresses to the mean

What Is the Logic behind it



The famous Math
equation

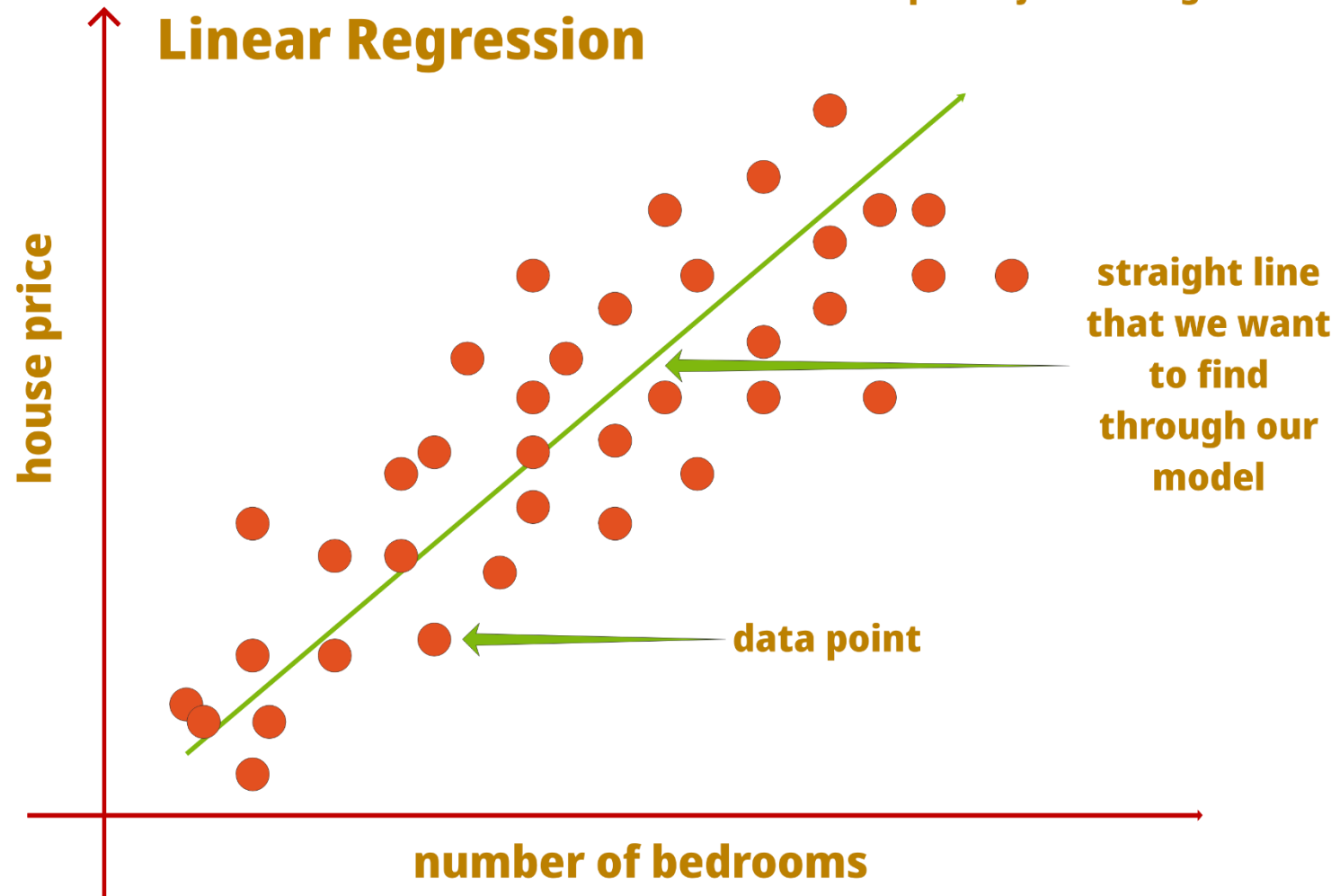
→ $y = ax + b$

Goal: is to find the best fitting linear line that
minimizes the difference between actual & predicted
values of y

Simplest Algorithm: Linear Regression

Regresses to the mean

<https://arjun-mota.github.io>



Simplest Algorithm: Linear Regression

1. Univariate Linear Regression (one feature)

- **Formula:** $y = ax + b$
- **Meaning**
 - $x \rightarrow$ one input feature (e.g., study hours)
 - $a \rightarrow$ how much x affects y
 - $b \rightarrow$ intercept (baseline value)
- **Example**
 - Predict math score using **study hours only**
If:
 - study hours = 5
 - model: $y = 4x + 40$Then:
 - Predicted score = $4 \times 5 + 40 = 60$

2. Multivariate Linear Regression (multiple features)

- **Formula:** $y = a_1x_1 + a_2x_2 + a_3x_3 + \dots + b$
- **Meaning**
 - Each x is a different feature (reading score, writing score, gender, lunch type, etc.)
 - Each feature has its **own weight**
 - Model combines **all features together**
- **Example**
 - Predict math score using:
 - reading score
 - writing score
 - gender
 - lunch type
 - test preparation
 - The model learns:
 - How much **each feature contributes** to the final score



**Let us run our first model using
Linear Regression**



Installation

The screenshot displays the Visual Studio Code interface with the Extensions Marketplace open. The search bar at the top left contains the text "jupyter". The extension list on the left shows several Jupyter-related extensions, with the "Jupyter" extension by Microsoft at the top, highlighted by an orange box. The extension's details are shown on the right, including its logo, name, publisher (Microsoft), version (2025.9.1), and a list of related extensions. The "Install" button for the "Jupyter" extension is also highlighted with an orange box. The extension's description states: "Jupyter notebook support, interactive programming and computing that supports Intellisense, debugging and more." The extension is also listed in the "Extension Pack (4)" section, which includes "Jupyter Slide Show", "Jupyter Keymap", "Jupyter Notebook ...", and "Jupyter PowerToys". The "Marketplace" section on the right shows the extension's identifier, version, published date, and last release date. The "Categories" section lists "Extension Packs", "Data Science", "Machine Learning", "Visualization", and "Notebooks". The "Resources" section includes links to the "Repository", "Issues", "License", "Microsoft", and "Marketplace". The bottom of the screen shows a code editor with a Python script and a terminal window.

Jupyter
Microsoft | 100,420,983 | ★★★★★ (359)
Jupyter notebook support, interactive programming and computing that supports Intellisense, debugging and more.

Extension Pack (4)

- Jupyter Slide Show**
Jupyter Slide Show support for VS Code | Microsoft | Install
- Jupyter Keymap**
Jupyter keymaps for notebooks | Microsoft | Install
- Jupyter Notebook ...**
Renderers for Jupyter Notebooks (w...) | Microsoft | Install
- Jupyter PowerToys**
Experimental features for Jupyter n... | Microsoft | Install

Marketplace

Identifier	ms-toolssai.jupyter
Version	2025.9.1
Published	5 years ago
Last Released	2 weeks ago

Categories

- Extension Packs
- Data Science
- Machine Learning
- Visualization
- Notebooks

Resources

- Repository
- Issues
- License
- Microsoft
- Marketplace

Jupyter Extension for Visual Studio Code

A Visual Studio Code extension that provides basic notebook support for language kernels that are supported in Jupyter Notebooks today, and allows any Python environment to be used as a Jupyter kernel. This is **NOT a Jupyter kernel**--you must have Python environment in which you've installed the Jupyter package, though many language kernels will work with no modification. To enable advanced features, modifications may be needed in the VS Code language extensions.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sys
```

1. Imports



```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

Numpy => math helper library

Pandas => reads csv and works like excel tables

Metric => to measure how good the model is

Matplotlib / seaborn => Graphs

To install them, do the following commands in the cmd

```
pip install numpy
pip install pandas
pip install matplotlib
pip install seaborn
pip install scikit-learn
```

Why do we import these libraries?

1. NumPy (numpy as np)

What it does: Fast math and numerical calculations

 **Real life example:**

Imagine you're calculating **average house prices**, **square meters**, or **loan interest** for thousands of houses. Doing this one by one is slow — NumPy lets you calculate everything **at once**, like a calculator for big data.

Why do we import these libraries?

2. Pandas (pandas as pd)

What it does: Reads data and organizes it like Excel

 **Real life example:**

Think of a **CSV file** as an Excel sheet with columns like:

- House size
- Number of rooms
- Location
- Price

Pandas lets us **load, clean, filter, and analyze** this table easily — just like Excel, but smarter and faster

Why do we import these libraries?

3. Matplotlib & Seaborn (matplotlib.pyplot, seaborn)

What they do: Create graphs and visualizations



Real life example:

Instead of saying “*bigger houses cost more*”, we **show it visually**:

- Scatter plot → house size vs price
- Line → predicted price trend

Seaborn makes graphs **beautiful**, Matplotlib gives **full control**.

Why do we import these libraries?

4. Linear Regression (LinearRegression)

What it does: Learns patterns and makes predictions

 **Real life example:**

You show the model past house data:

- “When size = 100m^2 → price $\approx$ \$200k”

The model learns this relationship and can now answer:

- “If a house is 120m^2 , what should the price be?”

This is exactly how **price prediction**, **salary estimation**, or **sales forecasting** works.

Why do we import these libraries?

5. Metrics (`r2_score`, `mean_absolute_error`, `mean_squared_error`)

What they do: Measure how good our predictions are



Real life example:

If the real house price is **\$300k** and the model predicts **\$290k**:

- **Error** = \$10k

Metrics answer questions like:

- How wrong are we on average?
- Is the model reliable enough to use in real life?

Think of it as **grading the model like an exam**

2. Load the dataset



```
df = pd.read_csv('stud.csv')
```

What this line does?

This line loads the student data from a CSV file into a table so we can analyze and build a model from it.

3. Preview the data



```
df.head( )
```

Displays the first 5 rows to quickly understand the data. **Used** to quickly inspect the structure and values of the dataset.

4. Creating X and previewing it



```
X = df.drop(columns = ['math_score'], axis=1)
```

Why we choose this as the output (target)?

“math_score” is the **result** we want the model to predict, so it cannot be used as an input

What this line does as small explanation?

Creates the **input data** (X) by removing the value we want to predict.

5. Handling categorical and numerical features

```
num_features = X.select_dtypes(exclude="object").columns
cat_features = X.select_dtypes(include="object").columns

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

numeric_transformer = StandardScaler()
oh_transformer = OneHotEncoder()

preprocessing= ColumnTransformer(
    [
        ("OneHotEncoder", oh_transformer, cat_features),
        ("StandardScaler", numeric_transformer, num_features),
    ]
)
```

OneHotEncoder=> 0/1
StandardScale=> -1, 0, 1

**Linear Regression can not use words directly =>
we must convert them into numbers**

5. Handling categorical and numerical features

what this does 🧐

Separates text and numeric columns, then prepares them so the model can understand them.

Why this step is needed?

Linear Regression cannot work with text, only numbers — so we convert everything into numeric form.

5. Handling categorical and numerical features

Explain each part:

1. Categorical (text) columns

Example: gender, school, lunch_type

- **OneHotEncoder converts text categories into 0s and 1s**
- **Example:**
 - Gender = Male $\rightarrow$ 1
 - Gender = Female $\rightarrow$ 0

This lets the model compare categories **mathematically** because everything depends on math.

5. Handling categorical and numerical features

Explain each part:

2. Numerical columns

Example: study_time, absences

- **StandardScaler puts numbers on the same scale** (around -1 to 1)
- **Why?**
 - So one big number doesn't dominate smaller ones
 - Makes the model learn **fairly and faster**

5. Handling categorical and numerical features

Explain each part:

3. ColumnTransformer

- **Applies the right transformation to each column automatically.**
- **Real-life analogy:**
 - Like translating different languages at the same time — text gets translated one way, numbers another.

So, We convert text to numbers and scale numeric values so the model can learn correctly.

6. Apply Transformation



```
X = preprocessor.fit_transform(X)
```

Like learning grading rules first, then grading all students the same way. Same thing for the model.

In the previous step the model learns how to encode and scale the data as we talk before, then it applies those rules to **transform X** into a model-ready format.

7. Create y (the output/target)



```
y = df['math_score']
```

y is what we want to predict

We here select the math score as the output we want the model to predict.

8. Splitting into training and testing

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

80% training: what the model will learn from

20% testing: where we check how good the model is on a new student

Random_state: use a specific random pattern for splitting, so every time we run it, the split stay the same
(42 is the most popular)

8. Splitting into training and testing

What this step is done for?

This step **splits** the data into **training data to learn from** and **testing data to evaluate** how well the model performs on new, unseen students.

You will all ask yourself why 80 - 20 not 50 – 50 🤔

With **50/50**, the model would **lose too much training data** and **learn poorly**, while **80/20** keeps **enough data for learning** and still **provides a reliable test set**.

9. Evaluating how good the model is



```
def evaluate_model(true, predicted):  
    mae = mean_absolute_error(true, predicted)  
    mse = mean_squared_error(true, predicted)  
    rmse = np.sqrt(mean_squared_error(true, predicted))  
    r2_square = r2_score(true, predicted)  
    return mae, rmse, r2_square
```

This step measures how accurate the model's predictions are compared to the real values.

9. Evaluating how good the model is

➤ MAE – Mean Absolute Error

- On average, the model is wrong by how many marks.
- If MAE = 3 → the model is wrong by about **3 marks** on average

➤ RMSE – Root Mean Squared Error

- Similar to MAE, but gives more weight to large mistakes.
- Why it matters: Penalizes predictions that are **very wrong**, not just slightly wrong

➤ R^2 – R-squared score

- Shows how well the model explains the pattern in the data (0 to 1).
- **Interpretation:**
 - 0 → model explains nothing
 - 1 → perfect predictions
 - 0.75 → explains **75% of the variation** in scores

10. Training Linear regression



```
model = LinearRegression(fit_intercept = True)
model.fit(X_train, y_train)
```

It tries to find the **best equation**

Learns how each **feature affects** the **math score**

So => This step trains the Linear Regression model to learn the relationship between the input features and the math score.

10. Predict Performance



```
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_mae, train_rmse, train_r2 = evaluate_model(y_train, y_train_pred)
test_mae, test_rmse, test_r2 = evaluate_model(y_test, y_test_pred)
```

This step uses the trained model to **make predictions** on both training and test data, then **compares** their **performance** to check if the model generalizes well.

10. Predict Performance

- **Training predictions** → checks how well the model learned the data it saw
- **Testing predictions** → checks how well the model works on new, unseen students

- If **training results** are **good** but **testing results** are **bad**, the model is **overfitting**.
- If **both** are **good**, the model **learned the real pattern - not just the data**.

11. Check Accuracy

```
score = r2_score(y_test, y_test_pred)*100  
print(" Accuracy of the model is %.2f" %score)
```

This step uses the **R² score** (not classification accuracy) on test data and displays it as a percentage to show how well the model explains student scores.

Interpretation:

- **0%** → model explains nothing
- **100%** → perfect prediction
- **75%** → model explains **75% of the variation** in math scores

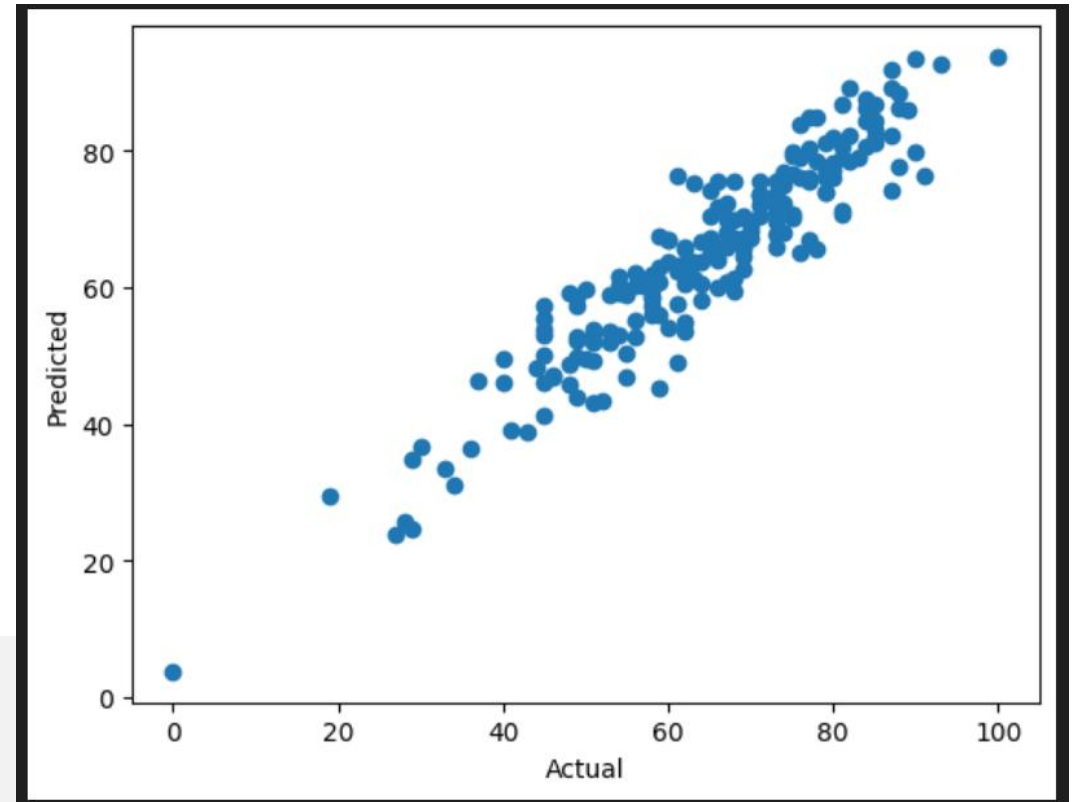
12. Plots (visual Check)

```
plt.scatter(y_test, y_test_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')
```

x_axis = real math score

y_axis = predicted math score

If dots are close to the line => our prediction is good

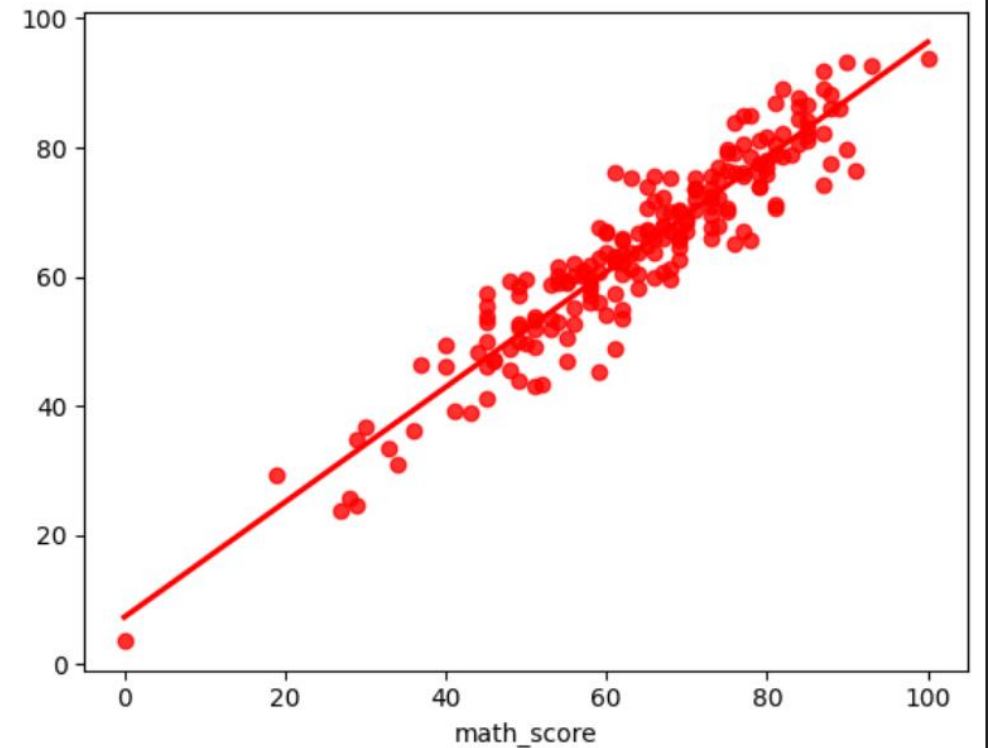


12. Regression Line Plot



```
sns.regplot(x=y_test,y=y_test_pred,ci=None,color='red');
```

To draw the best fit line



13. Table of Actual vs Predicted



```
pred_df=pd.DataFrame({'Actual Value':y_test,'Predicted Value':y_test_pred,'Difference':y_test-y_test_pred})  
pred_df
```

Shows **real vs predicted** values and the **difference** between them.

Meaning- for example: the model predicted 66 but the student actually scored 70 (**diff is 4**)

14. Prediction of a new student

```
new_student = pd.DataFrame({  
    "gender": ["female"],  
    "race_ethnicity": ["group C"],  
    "parental_level_of_education": ["associate's degree"],  
    "lunch": ["standard"],  
    "test_preparation_course": ["none"],  
    "reading_score": [66],  
    "writing_score": [67],  
    "math_score": [59]  
})
```

This step uses the trained model to predict the math score of a new student based on their information

14. Prediction of a new student



```
X_new = new_student.drop(columns=["math_score"])
X_new_processed = preprocessor.transform(X_new)
```



```
predicted_math_score = model.predict(X_new_processed)[0]
predicted_math_score
```



```
actual_math_score = new_student["math_score"].values[0]
difference = actual_math_score - predicted_math_score
```

```
comparison = pd.DataFrame({
    "Actual Math Score": [actual_math_score],
    "Predicted Math Score": [predicted_math_score],
    "Difference": [difference]
})
```

```
comparison
```




Now let us try to use another
dataset for same model
How much will it differ 🤔



What differs only: Load the dataset



```
# df = pd.read_csv("Advertising.csv", index_col=False) used to force  
pandas to not use the first column as the index  
df = pd.read_csv("Advertising.csv")
```

What differs only: Drop first unnecessary column



```
df.drop(df.columns[0], axis=1, inplace=True)  
df
```

The column is only about the
index (numbering) of each row

What differs only: Creating X and previewing it



```
X = df.drop(columns=["sales"], axis=1)
```

What differs only: Handling numerical features



```
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer

numerical_transformer = StandardScaler()

preprocessor = ColumnTransformer ([
    ("StandardScaler", numerical_transformer, X.columns)
])
```

Here in this data, we don't have categorical features, only we have numerical so we do **StandardScaler()**

What differs only: Create y (the output/target)



```
y = df["sales"]
```

What differs only: Prediction of a new data



```
new_advertise = pd.DataFrame({  
    "TV": [199.8],  
    "radio": [2.6],  
    "newspaper": [21.2],  
    "sales": [10.6]  
})
```



```
X_new = new_advertise.drop(columns=["sales"])  
new_advertise_processed = preprocessor.transform(X_new)
```

What differs only: Prediction of a new data



```
predicted_sales = model.predict(new_advertise_processed)[0]  
predicted_sales
```



```
actual_sales = new_advertise["sales"].values[0]  
difference = actual_sales - predicted_sales  
  
new_advertise_result = pd.DataFrame({  
    "Actual Sales": [actual_sales],  
    "Predicted Sales": [predicted_sales],  
    "Difference": [difference]  
})  
  
new_advertise_result
```




See You Saturday at 31 – 1 –
2026 🎂